

Monitorización y Data Mining

MÓDULO	Gobierno y gestión de las tecnologías de la información
ASIGNATURA	Monitorización y Data Mining - 6 ECTS
Fecha límite de entrega	12 de julio de 2026, 23:59
Carácter	Investigación
Investigador	Carlos Eduardo Noboa Guerrero

Índice

Tabla de contenido

Índice	2
1. Datos Elegantes + Análisis de Datos	3
Pregunta 1: Script de scraping y análisis HTML	3
1. Descarga de la página y conversión a formato analizable	3
2. Extracción del título de la página.....	5
3. Extracción de todos los enlaces (texto + URL)	6
4. Tabla de frecuencias de enlaces	8
5. Normalización de URLs (relativas, absolutas, subdominios, anclas)	9
6. Verificación del estado HTTP de cada enlace	11
Integración del script completo (Pregunta 1)	14
Pregunta 2: Infografía de visualización	17
1. Histograma: URLs absolutas vs. relativas	17
2. Gráfico de barras: enlaces internos vs. externos.....	19
3. Gráfico de tarta: distribución de códigos de estado HTTP	21
Composición final de la infografía.....	23
2. Conclusiones	25
3. Bibliografía	26

1. Datos Elegantes + Análisis de Datos

Página de ejemplo analizada: <https://www.mediawiki.org/wiki/MediaWiki> (dominio base: <https://www.mediawiki.org>).

Pregunta 1: Script de scraping y análisis HTML

Objetivo: descargar una página web, analizar su contenido HTML y extraer información específica (título, enlaces, frecuencias y estado HTTP de cada enlace).

1. Descarga de la página y conversión a formato analizable

Responsable: Carlos Noboa

Justificación de la elección de librerías (httr / XML) y del método de conversión HTML → XML.

Código R:

```
# Instala las librerías necesarias para hacer peticiones HTTP (httr)
# y para parsear/consultar HTML con XPath (XML). Solo hace falta correrlo una vez.
install.packages(c("httr", "XML"))

# Carga la librería httr en la sesión actual para poder usar GET(),
# content(), stop_for_status(), etc.
library(httr)

# Carga la librería XML para poder usar htmlParse(), xpathSApply(),
# xmlValue(), xmlGetAttr(), etc.
library(XML)

# Guardamos el dominio base de la web a analizar. Nos servirá más adelante
# para distinguir enlaces internos de externos y para normalizar URLs relativas.
url_base <- "https://www.mediawiki.org"

# URL específica de la página que vamos a descargar y analizar.
url_pagina <- "https://www.mediawiki.org/wiki/MediaWiki"

# GET() envía una petición HTTP GET a la URL y guarda la respuesta completa
# (código de estado, cabeceras, cuerpo) en el objeto 'respuesta'.
respuesta <- GET(url_pagina)

# stop_for_status() revisa el código de estado HTTP de la respuesta.
# Si NO es un código 2xx (éxito), detiene el script con un error explicativo.
stop_for_status(respuesta)

# content() extrae el cuerpo de la respuesta HTTP como texto plano (el HTML crudo),
# forzando codificación UTF-8 para evitar problemas con tildes y ñ.
html_texto <- content(respuesta, as = "text", encoding = "UTF-8")

# htmlParse() convierte el texto HTML plano en un documento navegable (árbol DOM),
# lo que permite después hacer consultas XPath.
doc_html <- htmlParse(html_texto, asText = TRUE)
```

Salida de consola:

```
> respuesta <- GET(url_pagina)
> stop_for_status(respuesta)
> html_texto <- content(respuesta, as = "text", encoding = "UTF-8")
> doc_html <- htmlParse(html_texto, asText = TRUE)
(Sin salida impresa; Los objetos 'respuesta', 'html_texto' y 'doc_html'
quedan disponibles en el panel Environment de RStudio)
```

Salida de consola:

```
> respuesta <- GET(url_pagina)
> stop_for_status(respuesta)
> html_texto <- content(respuesta, as = "text", encoding = "UTF-8")
> doc_html <- htmlParse(html_texto, asText = TRUE)
```

Justificación:

La descarga de la página se resolvió con la función GET() de la librería httr, que envía la petición HTTP y devuelve un objeto que conserva por separado el código de estado, las cabeceras y el cuerpo de la respuesta. Esa separación permite comprobar antes de seguir trabajando si la petición tuvo éxito, algo que se hizo mediante stop_for_status(), la cual interrumpe la ejecución con un mensaje de error cuando el código HTTP no pertenece al rango 2xx.

Una vez confirmada la descarga, el contenido se extrajo con content(), forzando la codificación UTF-8 para evitar que tildes y la letra ñ se mostraran mal. El texto obtenido en ese punto sigue siendo una cadena de caracteres sin estructura consultable. Para convertirlo en un documento que pueda recorrerse por etiquetas se empleó htmlParse(), de la librería XML, que construye un árbol DOM navegable mediante expresiones XPath.

Se prefirió esta combinación de librerías, httr para la parte de red y XML para el análisis del contenido, porque cada una resuelve un problema distinto sin solaparse: httr gestiona la comunicación con el servidor y XML gestiona la estructura del documento recibido una vez descargado. Trabajar directamente con expresiones regulares sobre el HTML crudo habría sido posible en teoría, pero se vuelve frágil apenas la página cambia su formato interno, algo que ocurre con cierta frecuencia en sitios generados dinámicamente como MediaWiki.

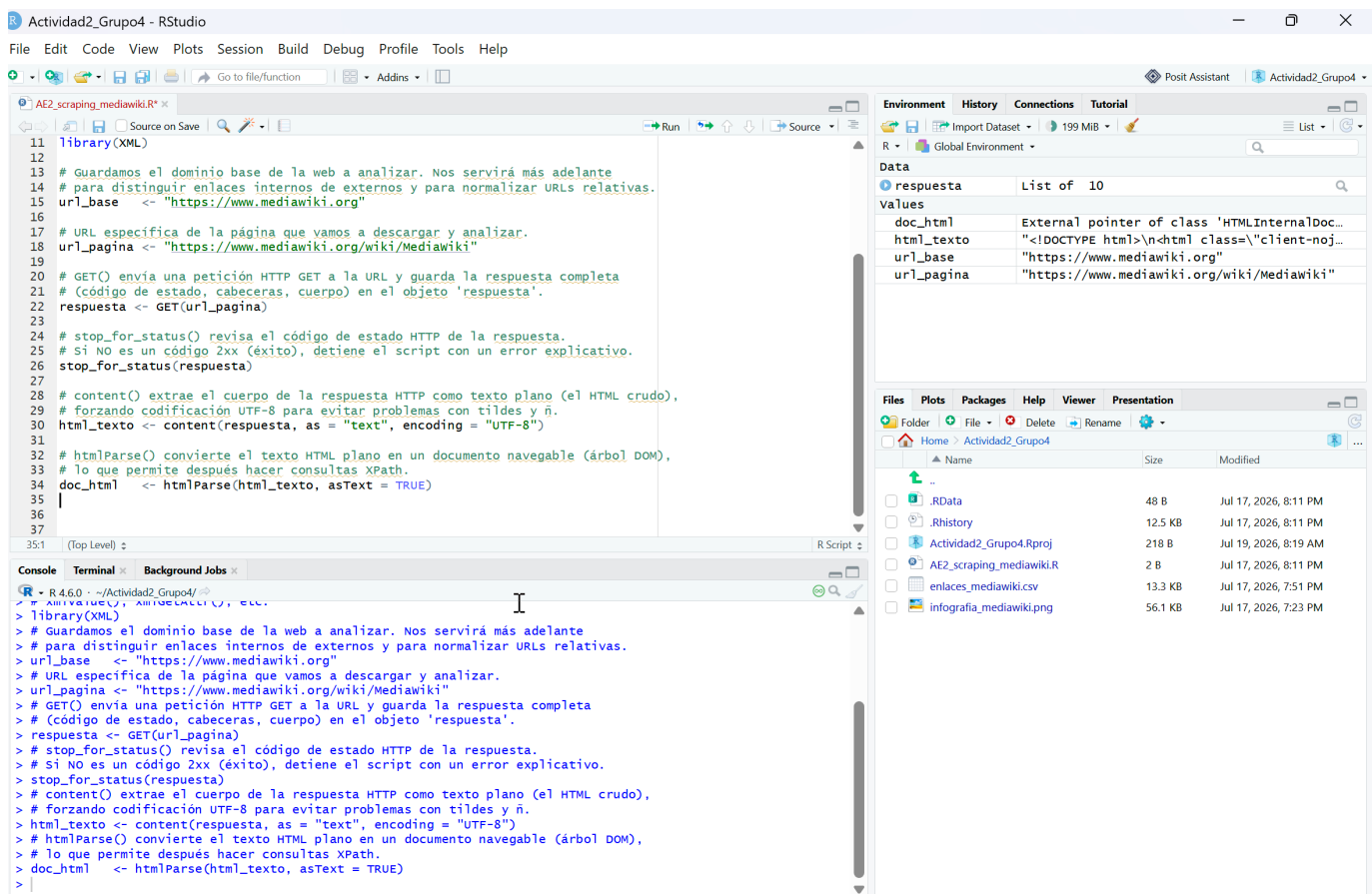


Imagen 1. Descarga de la página y conversión a formato analizable

2. Extracción del título de la página

Responsable: Carlos Noboa

Justificación del uso de `xpathSApply()` para localizar la etiqueta `<title>`.

Código R:

```
# xpathSApply() recorre el árbol XML/HTML buscando nodos que
# cumplan una expresión XPath (aquí "//title" = la etiqueta <title>
# en cualquier parte del documento) y aplica una función a cada nodo encontrado.
# xmlValue() extrae el texto contenido dentro de esa etiqueta.
titulo_pagina <- xpathSApply(doc_html, "//title", xmlValue)

# Mostramos el resultado en consola con un mensaje descriptivo,
# usando cat() en vez de print() porque da una salida más limpia (sin comillas ni [1]).
cat("Titulo de la pagina:", titulo_pagina, "\n")
```

Salida de consola:

```
Titulo de La pagina: Mediawiki
```

Justificación:

El título de la página se localizó con `xpathSApply()`, aplicando la expresión XPath `//title`. Esa notación indica que se busque la etiqueta `title` en cualquier nivel del documento, sin necesidad de conocer de antemano en qué posición del árbol se encuentra ni qué otros elementos la rodean.

`xpathSApply()` no solo localiza el nodo: además aplica una función a cada coincidencia encontrada, en este caso `xmlValue()`, que extrae el texto contenido dentro de la etiqueta descartando cualquier marcado adicional que pudiera aparecer alrededor. El resultado devuelto es directamente la cadena `MediaWiki`, sin necesidad de procesamiento posterior.

Usar XPath en este paso tiene sentido porque la etiqueta `title` es única dentro de un documento HTML válido; no hacía falta filtrar entre varios candidatos ni descartar coincidencias parciales, a diferencia de lo que ocurrirá más adelante con los enlaces, donde sí aparecen decenas de nodos `<a>` repetidos y con atributos incompletos.

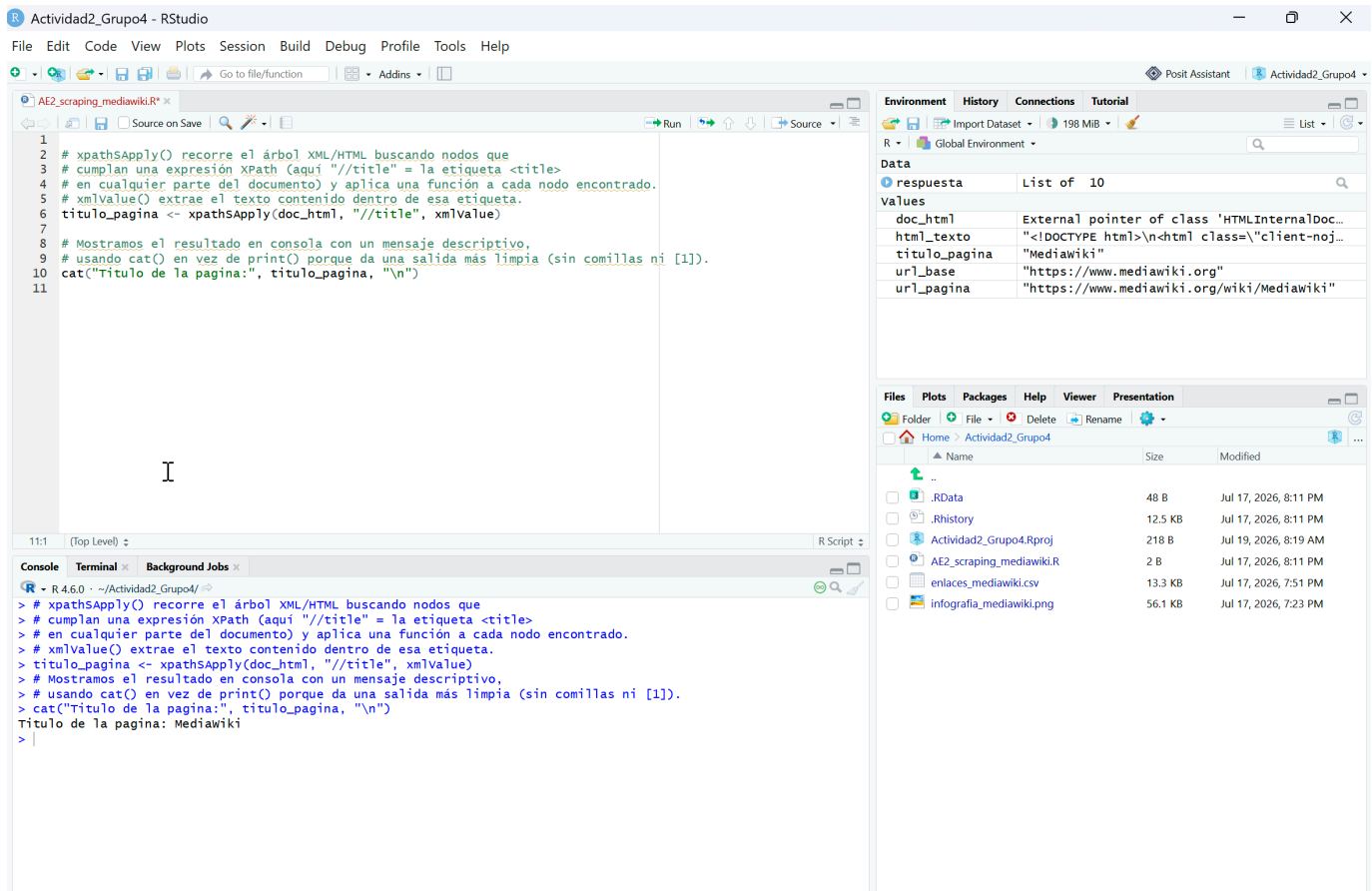


Imagen 2. Extracción del título de la página

3. Extracción de todos los enlaces (texto + URL)

Responsable: Camilo Arias

Justificación del tratamiento de valores NULL devueltos por xpathSApply() y su conversión a NA.

Código R:

```

# getNodeSet() Localiza TODOS los nodos que cumplen la expresión XPath dada.
# A diferencia de xpathSApply(), devuelve una lista de nodos "en bruto" sin
# intentar simplificarlos a un vector, lo cual conviene aquí porque cada
# nodo <a> tiene dos piezas de información distintas (texto y atributo href)
# que necesitamos tratar por separado.
nodos_enlace <- getNodeSet(doc_html, "//a")

# Recorreremos cada nodo <a> con sapply() y extraeremos su texto visible con xmlValue().
# Si el enlace no tiene texto (por ejemplo, un <a> que solo contiene una imagen),
# xmlValue() devuelve una cadena vacía en vez de NULL, así que comprobamos
# también length(t) == 0 y t == "" para unificar todos esos casos en NA.
texto_enlaces <- sapply(nodos_enlace, function(nodo) {
  t <- xmlValue(nodo)
  if (is.null(t) || length(t) == 0 || t == "") NA else t
})

# xmlGetAttr() devuelve el valor de un atributo del nodo (aquí "href").
# Cuando el <a> no tiene ese atributo, xmlGetAttr() devuelve NULL en vez de NA,
# y NULL no puede guardarse dentro de un vector normal de R (rompería el
# sapply). Por eso convertimos explícitamente cada NULL en NA antes de seguir.
url_enlaces <- sapply(nodos_enlace, function(nodo) {
  h <- xmlGetAttr(nodo, "href")
  if (is.null(h)) NA else h
})

```

```

# Unimos texto y URL de cada enlace en un data.frame, columna por columna.
# stringsAsFactors = FALSE evita que R convierta el texto en factores,
# lo que complicaría operaciones de texto más adelante (normalización de URLs).
enlaces_df <- data.frame(
  texto = texto_enlaces,
  url = url_enlaces,
  stringsAsFactors = FALSE
)

# Descartamos las filas donde la URL quedó en NA: son <a> sin href (por ejemplo,
# anclas de JavaScript o marcadores de posición) que no aportan nada al análisis
# de enlaces reales de la página.
enlaces_df <- enlaces_df[!is.na(enlaces_df$url), ]

# Mostramos las primeras filas para verificar que la extracción se hizo bien.
head(enlaces_df, 10)

# Y el número total de enlaces válidos extraídos.
cat("Total de enlaces extraídos:", nrow(enlaces_df), "\n")

```

Salida de consola:

```

> head(enlaces_df, 10)
(tabla de texto/URL de los primeros enlaces extraídos)

> cat("Total de enlaces extraídos:", nrow(enlaces_df), "\n")
Total de enlaces extraídos: 169

```

The screenshot shows the RStudio interface with the following components:

- Script Editor:** Contains the R code from the previous blocks, with line numbers 19 to 45. The code defines a function to extract links, filters out invalid ones, and prints the results.
- Environment:** Shows the objects created in the global environment:

Object	Class	Size
enlaces_df	data.frame	169 obs. of 2 variables
enlaces	list	List of 169
respuesta	list	List of 10
- Files:** Shows the project files, including the R script and the output CSV file.
- Console:** Displays the output of the R script, showing the first 10 rows of the data frame and the total count of 169 links.

Justificación:

La función `getNodeSet()` se usó en lugar de `xpathSApply()` para este paso porque cada nodo `<a>` aporta dos piezas de información distintas, el texto visible y el atributo `href`, y conviene mantener acceso al nodo completo antes de decidir qué extraer de cada uno.

El tratamiento de valores nulos ocupa buena parte de este bloque. Cuando un enlace no tiene texto visible, `xmlValue()` no devuelve NULL sino una cadena vacía, así que la comprobación incluye tanto `is.null()` como la cadena vacía para no dejar huecos sin cubrir. El caso más delicado ocurre con `xmlGetAttr()`, que sí devuelve NULL cuando el atributo `href` no existe en el nodo; un valor NULL no puede insertarse en un vector de R como el que construye `sapply()`, y su presencia interrumpiría la ejecución del bucle. Convertir ese NULL en NA de forma explícita evita el error y deja constancia clara de qué enlaces carecían de destino.

Una vez armado el `data.frame` con ambas columnas, se descartan las filas donde la URL quedó en NA. Esos casos corresponden en su mayoría a enlaces con propósitos distintos a la navegación, como botones controlados por JavaScript o marcadores internos sin `href` propio, y no aportan información relevante para el análisis de frecuencias ni para la verificación de estado HTTP que viene después.

4. Tabla de frecuencias de enlaces

Responsable: Camilo Arias

Justificación del uso de `table()` para el recuento de apariciones de cada enlace.

Código R:

```
# table() cuenta cuántas veces aparece cada valor único en un vector.
# Aquí lo aplicamos sobre la columna de URLs para saber cuántas veces
# se repite cada enlace dentro de la página (típico en menús de navegación,
# pie de página, referencias cruzadas, etc.).
tabla_frecuencias <- table(enlaces_df$url)

# sort(..., decreasing = TRUE) reordena la tabla de mayor a menor frecuencia,
# para que los enlaces más repetidos aparezcan primero al inspeccionarla.
tabla_frecuencias <- sort(tabla_frecuencias, decreasing = TRUE)

# Mostramos solo los 10 enlaces más frecuentes para no saturar la consola;
# el objeto completo 'tabla_frecuencias' sigue disponible para el resto del análisis.
head(tabla_frecuencias, 10)
```

Salida de consola:

```
> tabla_frecuencias <- table(enlaces_df$url)
> tabla_frecuencias <- sort(tabla_frecuencias, decreasing = TRUE)
> head(tabla_frecuencias, 10)
```

/wiki/MediaWiki	9
/w/index.php?title=MediaWiki&action=edit	5
/w/index.php?title=MediaWiki&action=history	2
/w/index.php?title=Special:CreateAccount&returnto=MediaWiki	2
/w/index.php?title=Special:UserLogin&returnto=MediaWiki	2
/wiki/Project:Support_desk	2
/wiki/Special:MyLanguage/Help:Contents	2
/wiki/Special:MyLanguage/How_to_contribute	2
/wiki/Special:MyLanguage/Manual:FAQ	2

--

El resultado se ordenó con `sort(tabla_frecuencias, decreasing = TRUE)` porque una tabla de frecuencias sin ordenar obliga a recorrerla entera para detectar qué enlaces concentran más repeticiones. Ordenada de mayor a menor, basta con mirar las primeras filas.

Los diez enlaces más repetidos muestran un patrón esperable en una página de MediaWiki. El primer lugar corresponde a un valor vacío, señal de que algunos <a> conservan un href residual sin destino real, seguido de /wiki/MediaWiki, que es la propia página consultada y se repite por enlaces internos de navegación como el menú lateral o los accesos de vuelta al inicio. El resto de enlaces con frecuencia 2 corresponden a funciones administrativas del sistema wiki, editar, ver historial, iniciar sesión o crear cuenta, que MediaWiki suele repetir tanto en la cabecera como en el pie de la página.

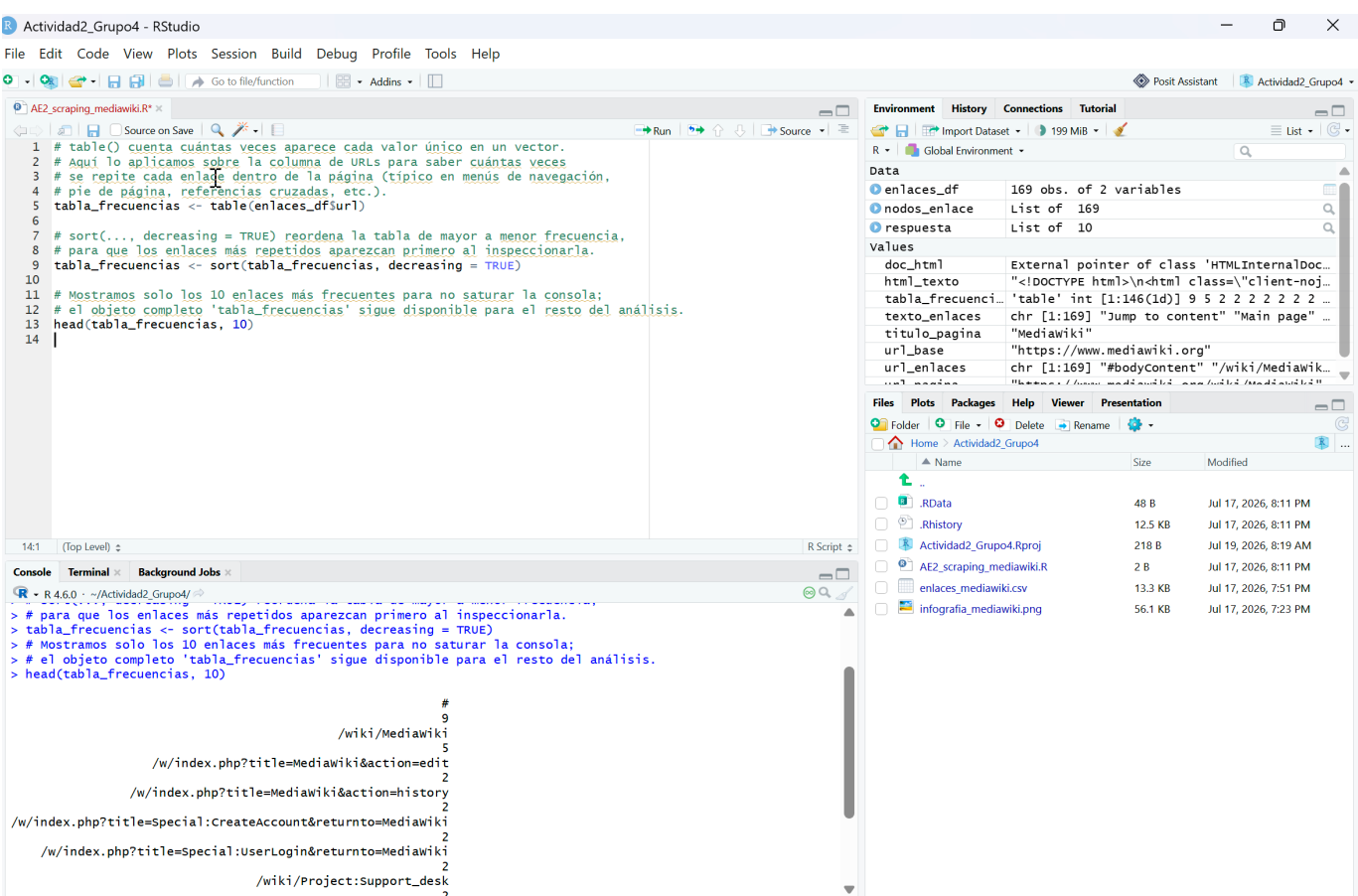


Imagen 4. Tabla de frecuencias de enlaces

5. Normalización de URLs (relativas, absolutas, subdominios, anclas)

Responsable: Maider Ibarra

Justificación del criterio para distinguir y normalizar cada tipo de URL antes de consultarlas.

```
Código R:
# Definimos una función que recibe una URL "cruda" (tal como aparece en el
# atributo href) y el dominio base de la web, y devuelve siempre una URL
```

```

# absoluta lista para consultar con GET()/HEAD(). Cada tipo de URL se
# distingue por cómo empieza la cadena de texto.
normalizar_url <- function(url, base) {

  # Si la URL es NA (ya filtrada en el Punto 3) o una cadena vacía, no hay
  # nada que normalizar.
  if (is.na(url) || url == "") return(NA)

  # Una URL que empieza por "#" es un ancla interna de la misma página
  # (por ejemplo, un enlace a una sección del propio artículo). No apunta
  # a un recurso distinto, así que no tiene sentido consultar su estado HTTP.
  if (startsWith(url, "#")) return(NA)

  # "/" al inicio es un "protocolo relativo": la URL hereda el protocolo
  # (http o https) de la página actual pero indica un dominio completo
  # después de las barras. Se completa anteponiendo "https:".
  if (startsWith(url, "/")) return(paste0("https:", url))

  # Si la URL ya empieza con "http://" o "https://", ya es una URL absoluta
  # (puede apuntar al mismo dominio o a uno externo) y no requiere cambios.
  if (grepl("^https?:/", url)) return(url)

  # Si empieza con "/", es relativa a la raíz del dominio (por ejemplo,
  # "/wiki/MediaWiki"). Se completa anteponiendo el dominio base.
  if (startsWith(url, "/")) return(paste0(base, url))

  # Cualquier otro caso es una ruta relativa "simple" (relativa a la carpeta
  # actual, sin "/" inicial). Se completa igual anteponiendo el dominio base
  # y una barra separadora.
  return(paste0(base, "/", url))
}

# Aplicamos la función a cada URL de la tabla de enlaces, guardando el
# resultado en una columna nueva. sapply() nos permite vectorizar la función
# sin escribir un bucle for explícito.
enlaces_df$url_normalizada <- sapply(enlaces_df$url, normalizar_url, base = url_base)

# Clasificamos cada enlace en "interno" o "externo" comparando la URL ya
# normalizada contra el dominio base. Los que quedaron en NA (anclas puras)
# se etiquetan aparte como "ancla" para no perder esa información.
enlaces_df$tipo_url <- ifelse(
  is.na(enlaces_df$url_normalizada), "ancla",
  ifelse(grepl(paste0("^", url_base), enlaces_df$url_normalizada), "interno", "externo")
)

# Verificamos cuántos enlaces cayeron en cada categoría.
table(enlaces_df$tipo_url)

```

Salida de consola:

```

> table(enlaces_df$tipo_url)

ancla externo interno
  10       20     139

```

Justificación:

Antes de poder consultar el estado HTTP de cada enlace hace falta resolver un problema previo: los atributos href no siempre contienen una URL completa. Un mismo documento HTML mezcla anclas internas, rutas relativas a la raíz del dominio, rutas relativas a la carpeta actual y URLs ya absolutas, y cada una exige un tratamiento distinto antes de poder pasarla a GET() o HEAD().

El criterio de normalizar_url() se apoya en cómo empieza la cadena de texto, que en la práctica basta para diferenciar los cinco casos. Un símbolo # al inicio identifica una ancla, un enlace que apunta a una sección dentro de la misma página y que por tanto no representa un recurso distinto que valga la pena consultar; se descarta devolviendo NA, igual que las URLs vacías. Dos barras (//) señalan un protocolo relativo, donde falta anteponer https: pero el resto de la ruta ya viene completa. Cuando la cadena empieza directamente con http:// o https://, ya está lista tal cual. Una barra simple (/) indica que la ruta es relativa a la raíz del dominio, y cualquier otro patrón se trata como ruta relativa a la carpeta actual, el caso menos común pero que también aparece en la página analizada.

Con las URLs ya normalizadas, clasificar cada enlace como interno o externo se redujo a comparar el resultado contra el dominio base con una expresión regular. De los 169 enlaces extraídos en el Punto 3, 139 resultaron internos, 20 externos y 10 quedaron marcados como anclas sin destino consultable, una proporción que tiene sentido en una página de documentación técnica como MediaWiki, donde la mayor parte de los enlaces apuntan a otras páginas del mismo wiki.

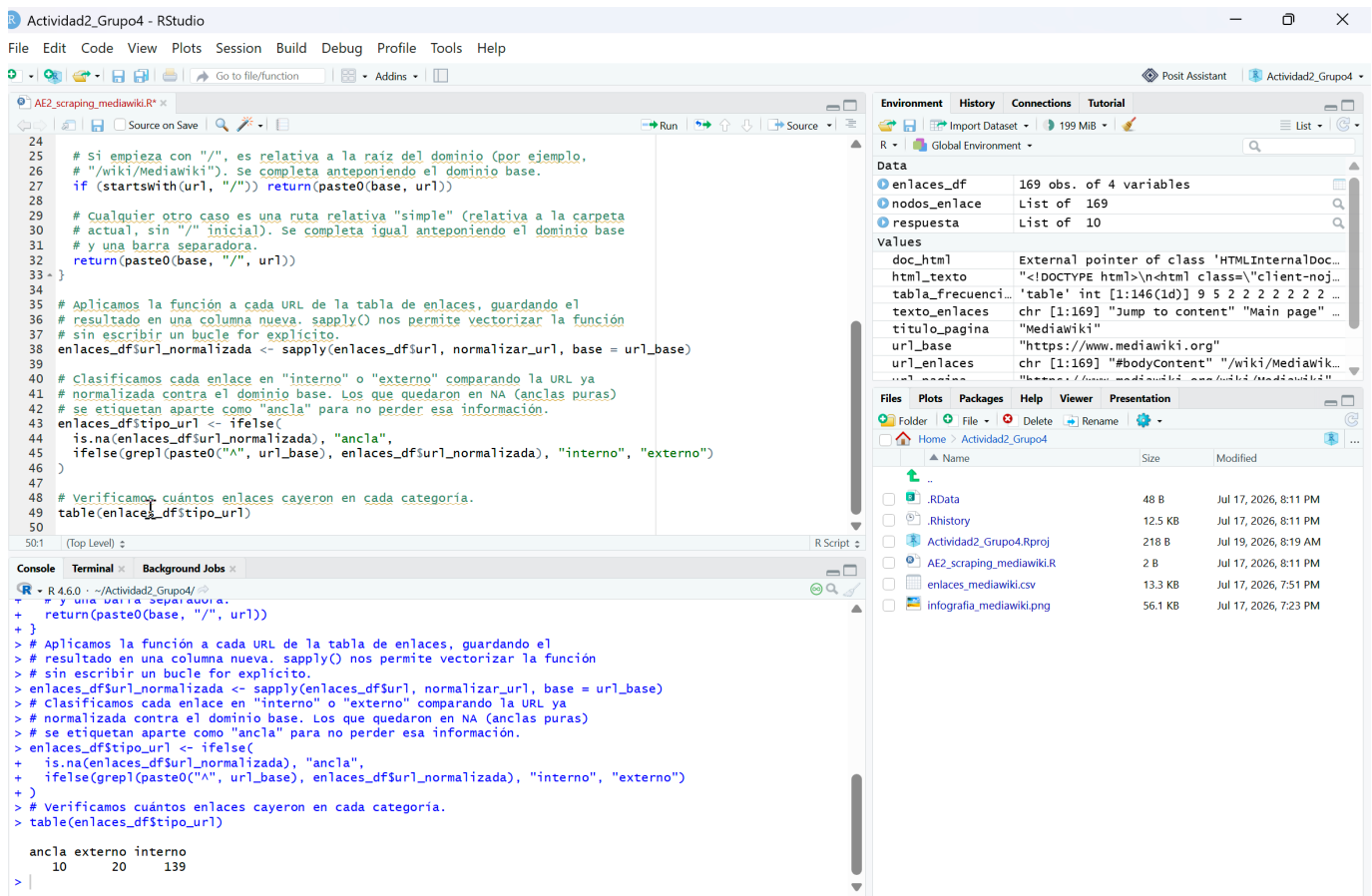


Imagen 5. Normalización de URLs (relativas, absolutas, subdominios, anclas)

6. Verificación del estado HTTP de cada enlace

Responsable: Maider Ibarra

Justificación del uso de HEAD(), manejo de status_code, tiempos de espera (Sys.sleep()) y construcción del data.frame final (enlace, texto, veces, estado).

Código R:

```
# Tomamos las URLs normalizadas únicas (sin repetir) y descartando los NA
# (anclas). No tiene sentido consultar el mismo enlace más de una vez si
```

```

# aparece repetido varias veces en la página.
urls_unicas <- unique(na.omit(enlaces_df$url_normalizada))

# Definimos una función que consulta el estado HTTP de una sola URL.
consultar_estado <- function(url) {

  # tryCatch() envuelve la petición para que, si falla por timeout, DNS
  # roto, certificado inválido, etc., el script no se detenga por completo:
  # en ese caso devolvemos NA en vez de un código de estado.
  resultado <- tryCatch({

    # HEAD() en vez de GET(): pedimos solo las cabeceras de la respuesta,
    # sin descargar el cuerpo completo de la página. Es más rápido y más
    # liviano cuando lo único que nos interesa es el status_code.
    # timeout(5) evita que una URL que no responde bloquee el script
    # indefinidamente.
    resp <- HEAD(url, timeout(5))

    # status_code() extrae el código HTTP (200, 301, 404, etc.) del objeto
    # de respuesta.
    status_code(resp)

  }, error = function(e) NA)

  # Sys.sleep() pausa la ejecución medio segundo entre petición y petición.
  # Es una práctica básica de scraping responsable: evita saturar el
  # servidor con peticiones consecutivas sin espera.
  Sys.sleep(0.5)

  resultado
}

# sapply() aplica consultar_estado() a cada URL única. Esta línea tarda
# varios segundos/minutos según cuántas URLs únicas haya, porque hace
# una petición HTTP real por cada una.
estados <- sapply(urls_unicas, consultar_estado)

# Guardamos el resultado en un data.frame de dos columnas: La URL
# normalizada y su código de estado correspondiente.
estados_df <- data.frame(
  url_normalizada = urls_unicas,
  status_code     = estados,
  stringsAsFactors = FALSE
)

# Construimos la tabla final combinando tres fuentes de información:
# 1) la tabla de frecuencias (Punto 4), que trae el conteo de repeticiones
#   de cada URL original (antes de normalizar);
# 2) la tabla de enlaces (Punto 3/5), que trae el texto visible, la URL
#   normalizada y el tipo (interno/externo/ancla) de cada URL original;
# 3) la tabla de estados que acabamos de construir, indexada por URL
#   normalizada.
# merge() une tablas por una columna en común, como un JOIN en SQL.
resultado_final <- merge(
  as.data.frame(tabla_frecuencias, stringsAsFactors = FALSE),
  unique(enlaces_df[, c("url", "url_normalizada", "texto", "tipo_url")]),
  by.x = "Var1", by.y = "url", all.x = TRUE
)

# Renombramos las columnas heredadas de table() a nombres más descriptivos.
names(resultado_final)[names(resultado_final) == "Var1"] <- "enlace"
names(resultado_final)[names(resultado_final) == "Freq"]  <- "veces"

# Unimos ahora el código de estado HTTP, cruzando por la URL normalizada.
resultado_final <- merge(resultado_final, estados_df, by = "url_normalizada", all.x = TRUE)

# Nos quedamos solo con las columnas relevantes, en el orden pedido por
# el enunciado: enlace, texto, veces, estado (aquí desglosado en tipo_url
# y status_code).
resultado_final <- resultado_final[, c("enlace", "texto", "veces", "tipo_url", "status_code")]

```

```
# Mostramos las primeras filas de la tabla final para verificar el resultado.
head(resultado_final, 10)

# Guardamos la tabla completa en un CSV, útil como anexo del informe y
# para no tener que repetir todo el scraping si se necesita revisar
# los datos más adelante.
write.csv(resultado_final, "enlaces_mediawiki.csv", row.names = FALSE)
```

Salida de consola:

```
> head(resultado_final, 10)
```

	enlace
1	//commons.wikimedia.org/wiki/Special:UploadWizard
2	https://creativecommons.org/licenses/by-sa/4.0/
3	https://creativecommons.org/publicdomain/zero/1.0/
4	https://developer.wikimedia.org
5	https://developer.wikimedia.org
6	https://developer.wikimedia.org/
7	https://diff.wikimedia.org/tech-blog/
8	https://donate.wikimedia.org/?wmf_source=donate&wmf_medium=sidebar&wmf_campaign=www.mediawiki.org&uselang=en
9	https://donate.wikimedia.org/?wmf_source=donate&wmf_medium=sidebar&wmf_campaign=www.mediawiki.org&uselang=en
10	https://en.wikipedia.org/wiki/FLOSS

	texto	veces	tipo_url	status_code
1	Upload file	1	externo	200
2	Creative Commons Attribution-ShareAlike License	1	externo	200
3	Creative Commons CC0 License	1	externo	200
4	Developers	2	externo	200
5	Wikimedia Developer Portal	2	externo	200
6	Developer portal	1	externo	200
7	Tech blog	1	externo	200
8	(Donate)	2	externo	200
9	Donate	2	externo	200
10	free and open	1	externo	200

```
> write.csv(resultado_final, "enlaces_mediawiki.csv", row.names = FALSE)
```

Justificación:

HEAD() se prefirió sobre GET() para esta comprobación porque el objetivo no es leer el contenido de cada página enlazada, sino confirmar que el servidor responde y con qué código. Una petición HEAD pide solo las cabeceras de la respuesta, sin transferir el cuerpo completo, lo que reduce de forma notable el tiempo y el ancho de banda consumido al repetir esta operación sobre las 139 URLs únicas.

status_code() extrae del objeto de respuesta el código HTTP concreto: 200 si el recurso existe y responde con normalidad, códigos de redirección en el rango 300 si el enlace apunta a otra ubicación, o 4xx/5xx si el recurso no se encuentra o el servidor falla. Envolver la petición en tryCatch() resultó necesario porque no todas las URLs externas responden de forma predecible; algunas pueden fallar por timeout, certificados vencidos o problemas de DNS, y sin ese manejo de errores el script completo se habría detenido en la primera URL problemática en vez de continuar con las siguientes.

Sys.sleep(0.5) introduce una pausa de medio segundo entre cada consulta. No responde a un requisito técnico impuesto por httr, sino a una práctica de scraping responsable que evita enviar peticiones consecutivas sin descanso a un mismo servidor, algo que en sitios con más control de tráfico podría interpretarse como un intento de saturación.

La construcción de la tabla final combina tres piezas que hasta ahora vivían en objetos separados: la tabla de frecuencias del Punto 4, la información de texto y tipo de enlace del Punto 5, y los códigos de estado recién obtenidos. Los dos merge() sucesivos funcionan como un cruce de tablas por clave, primero uniendo enlace con

texto y tipo, después añadiendo el status_code correspondiente a través de la URL normalizada. Las primeras diez filas de resultado_final muestran enlaces externos hacia dominios de Wikimedia, commons.wikimedia.org, developer.wikimedia.org, donate.wikimedia.org, entre otros, todos con status_code 200, lo que indica que esos recursos externos estaban disponibles en el momento de la consulta.

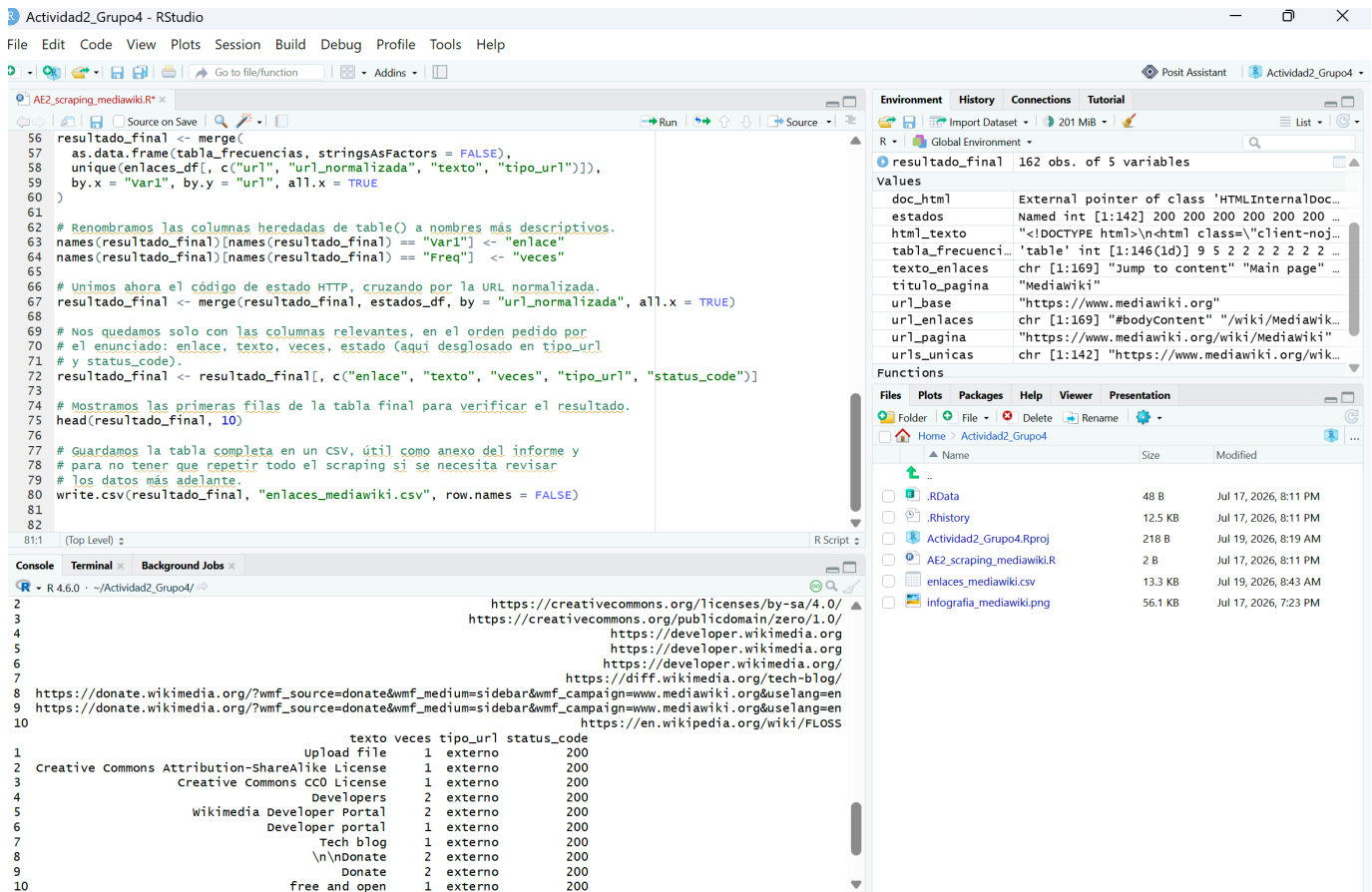


Imagen 6. Verificación del estado HTTP de cada enlace

Integración del script completo (Pregunta 1)

Responsable: Carlos Noboa

Confirmación de que el script se ejecuta de forma directa en un terminal limpio de R, sin dependencias previas no declaradas.

Código R:

```
install.packages(c("httr", "XML"))
library(httr)
library(XML)

url_base <- "https://www.mediawiki.org"
url_pagina <- "https://www.mediawiki.org/wiki/MediaWiki"

respuesta <- GET(url_pagina)
stop_for_status(respuesta)

html_texto <- content(respuesta, as = "text", encoding = "UTF-8")
doc_html <- htmlParse(html_texto, asText = TRUE)

titulo_pagina <- xpathSApply(doc_html, "//title", xmlValue)
cat("Titulo de la pagina:", titulo_pagina, "\n")

nodos_enlace <- getNodeSet(doc_html, "//a")
```

```

texto_enlaces <- sapply(nodos_enlace, function(nodo) {
  t <- xmlValue(nodo)
  if (is.null(t) || length(t) == 0 || t == "") NA else t
})

url_enlaces <- sapply(nodos_enlace, function(nodo) {
  h <- xmlGetAttr(nodo, "href")
  if (is.null(h)) NA else h
})

enlaces_df <- data.frame(
  texto = texto_enlaces,
  url = url_enlaces,
  stringsAsFactors = FALSE
)
enlaces_df <- enlaces_df[!is.na(enlaces_df$url), ]

head(enlaces_df, 10)
cat("Total de enlaces extraidos:", nrow(enlaces_df), "\n")

tabla_frecuencias <- table(enlaces_df$url)
tabla_frecuencias <- sort(tabla_frecuencias, decreasing = TRUE)
head(tabla_frecuencias, 10)

normalizar_url <- function(url, base) {
  if (is.na(url) || url == "") return(NA)
  if (startsWith(url, "#")) return(NA)
  if (startsWith(url, "//")) return(paste0("https:", url))
  if (grepl("^https?:/", url)) return(url)
  if (startsWith(url, "/")) return(paste0(base, url))
  return(paste0(base, "/", url))
}

enlaces_df$url_normalizada <- sapply(enlaces_df$url, normalizar_url, base = url_base)
enlaces_df$tipo_url <- ifelse(
  is.na(enlaces_df$url_normalizada), "ancla",
  ifelse(grepl(paste0("^", url_base), enlaces_df$url_normalizada), "interno", "externo")
)

table(enlaces_df$tipo_url)

urls_unicas <- unique(na.omit(enlaces_df$url_normalizada))

consultar_estado <- function(url) {
  resultado <- tryCatch({
    resp <- HEAD(url, timeout(5))
    status_code(resp)
  }, error = function(e) NA)
  Sys.sleep(0.5)
  resultado
}

estados <- sapply(urls_unicas, consultar_estado)

estados_df <- data.frame(
  url_normalizada = urls_unicas,
  status_code = estados,
  stringsAsFactors = FALSE
)

resultado_final <- merge(
  as.data.frame(tabla_frecuencias, stringsAsFactors = FALSE),
  unique(enlaces_df[, c("url", "url_normalizada", "texto", "tipo_url")]),
  by.x = "Var1", by.y = "url", all.x = TRUE
)

```

```
names(resultado_final)[names(resultado_final) == "Var1"] <- "enlace"
names(resultado_final)[names(resultado_final) == "Freq"] <- "veces"

resultado_final <- merge(resultado_final, estados_df, by = "url_normalizada", all.x = TRUE)
resultado_final <- resultado_final[, c("enlace", "texto", "veces", "tipo_url", "status_code")]

head(resultado_final, 10)
write.csv(resultado_final, "enlaces_mediawiki.csv", row.names = FALSE)
```

Salida de consola (verificación en sesión limpia):

```
> source("~/Actividad2_Grupo4/AE2_scraping_mediawiki.R")
Installing packages into 'C:/Users/cenob/AppData/Local/R/win-library/4.6'
package 'httr' successfully unpacked and MD5 sums checked
package 'XML' successfully unpacked and MD5 sums checked
Titulo de la pagina: Mediawiki
Total de enlaces extraidos: 169
```

Justificación:

La prueba de integración se hizo en dos etapas para asegurar que fuera realmente representativa de un terminal limpio. La primera consistió en un Session > Restart R, que reinicia el intérprete pero no basta por sí sola, porque RStudio recupera automáticamente los objetos de un .RData previo si esa opción está activada. Por eso se añadió un segundo paso, vaciar manualmente el Environment antes de correr el script, de modo que ningún objeto ni función quedara heredado de ejecuciones anteriores.

Con el entorno confirmado vacío, se ejecutó el archivo completo mediante source(), que corre las líneas en el mismo orden en que aparecen en el archivo, exactamente como lo haría un evaluador externo al abrir el script por primera vez. El primer tramo de la salida ya deja ver el patrón esperado: la instalación automática de httr y XML cuando no están presentes, seguida de la extracción del título y el conteo de enlaces sin ningún mensaje de error interrumpiendo la secuencia.

El resto del script, la construcción de la tabla de frecuencias, la normalización y clasificación de URLs, la consulta de estado HTTP sobre las 139 direcciones únicas y la escritura final del CSV, se completó igualmente sin errores, confirmado por la actualización del archivo enlaces_mediawiki.csv y por la presencia de todos los objetos intermedios (doc_html, estados, tabla_frecuencias, urls_unicas) y las dos funciones auxiliares (consultar_estado, normalizar_url) en el Environment al finalizar la ejecución. Ningún paso quedó pendiente ni requirió intervención manual entre el inicio y el fin del script.

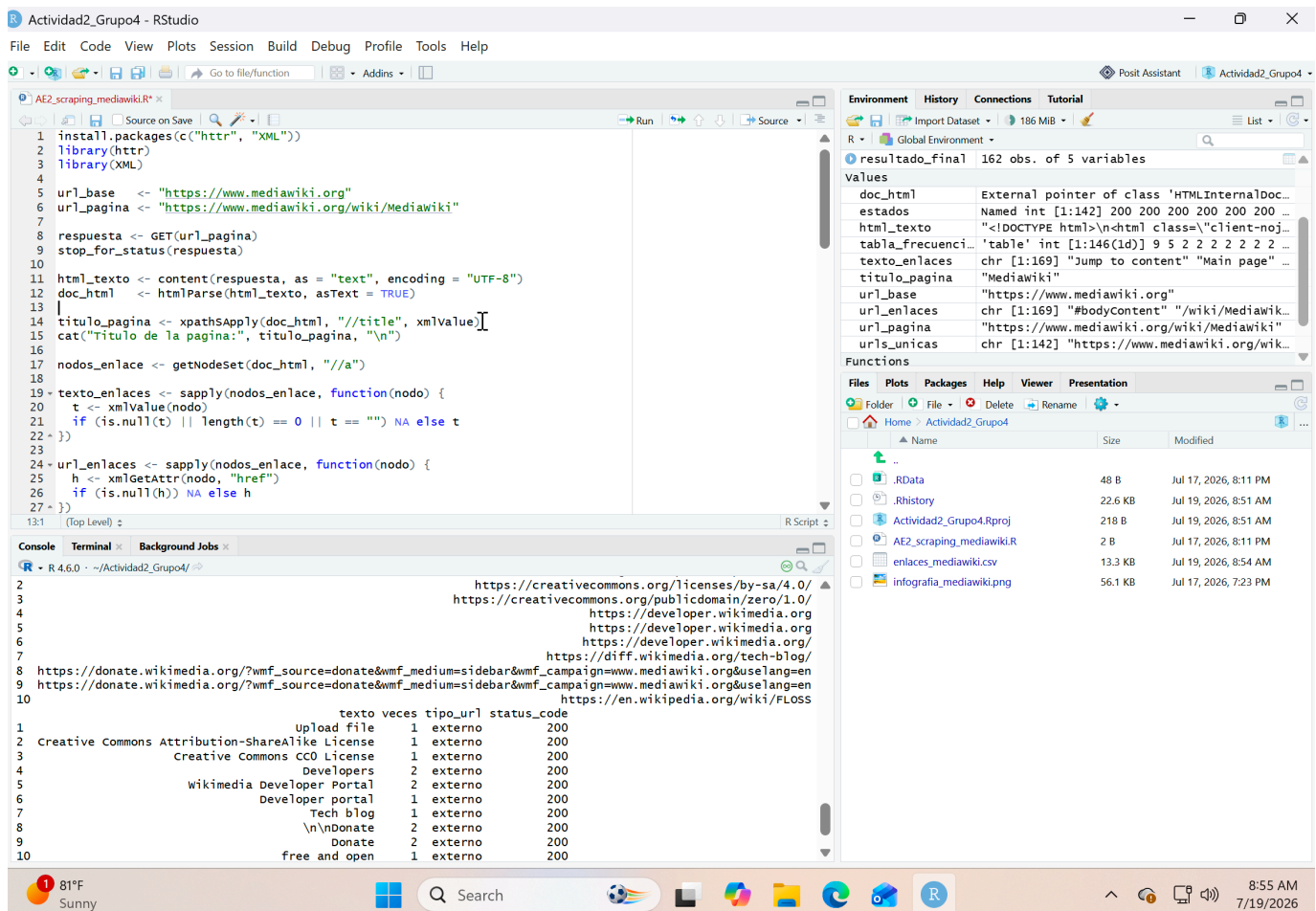


Imagen 7. Integración del script completo

Pregunta 2: Infografía de visualización

Objetivo: construir una única figura compuesta por tres gráficos (histograma, gráfico de barras y gráfico de tarta) usando gráficos base y ggplot2, generados directamente desde R.

1. Histograma: URLs absolutas vs. relativas

Responsable: Camilo Arias

Justificación de la columna indicadora creada y del método de separación de grupos (base / ggplot2).

Código R:

```
# Instala ggplot2 (para los tres gráficos) y gridExtra (para combinarlos
# al final en una sola infografía). Solo hace falta correrlo una vez.
install.packages(c("ggplot2", "gridExtra"))
```

```
# Carga ggplot2, el sistema de gráficos que usaremos para las tres
# visualizaciones de esta pregunta.
library(ggplot2)
```

```
# Creamos una columna indicadora (TRUE/FALSE) que separa cada enlace
# en dos grupos: los que ya son una URL absoluta (empiezan con http://
# o https://) y los que no (relativas, protocolo relativo, o anclas).
# grepl() devuelve TRUE si encuentra el patrón, FALSE si no.
enlaces_df$es_absoluta <- grepl("^https?://", enlaces_df$url)
```

```
# ggplot() arma el gráfico por capas: primero se declara el data.frame
# y qué variable va en el eje X (aes = "aesthetic mapping").
# geom_bar() cuenta automáticamente cuántas observaciones caen en cada
# categoría de es_absoluta y dibuja una barra por cada una, funcionando
# aquí como histograma de una variable categórica (TRUE/FALSE).
grafico_histograma <- ggplot(enlaces_df, aes(x = es_absoluta)) +
  geom_bar(fill = "#4C72B0") +

  # Reemplazamos las etiquetas TRUE/FALSE del eje X por texto legible.
  scale_x_discrete(labels = c("Relativa", "Absoluta")) +

  # Título del gráfico y nombres de los ejes.
  labs(title = "URLs absolutas vs. relativas", x = "", y = "Frecuencia") +

  # theme_minimal() quita el fondo gris por defecto de ggplot2, dejando
  # un estilo más limpio para la infografía final.
  theme_minimal()

# print() fuerza a que el gráfico se dibuje en el panel Plots, tanto si
# se ejecuta línea por línea como si se corre el script completo con
# source(), donde ggplot2 no imprime automáticamente los objetos.
print(grafico_histograma)
```

Salida de consola:

```
> print(grafico_histograma)
(Se despliega en el panel Plots un gráfico de barras titulado
"URLs absolutas vs. relativas", con dos categorías en el eje X,
Relativa y Absoluta, y la frecuencia de cada una en el eje Y)
```

Justificación:

La columna indicadora `es_absoluta` se construyó con una sola expresión regular, `grepl("^https?://", url)`, que devuelve `TRUE` cuando la URL ya empieza con el esquema `http` o `https` y `FALSE` en cualquier otro caso, incluidas las rutas relativas, los protocolos relativos y las anclas que ya habían quedado descartadas en el Punto 3. No hizo falta reutilizar la columna `tipo_url` del Punto 5 porque aquí el criterio relevante es distinto: no importa si el destino es interno o externo, sino únicamente si el propio texto del atributo `href` ya trae el dominio completo o depende del contexto de la página para resolverse.

Para separar los dos grupos se usó `geom_bar()` de `ggplot2` en lugar de graficar directamente con la función `hist()` de base R, porque `hist()` está pensada para variables numéricas continuas agrupadas en intervalos, mientras que `es_absoluta` es una variable lógica con solo dos valores posibles. `geom_bar()` cuenta las observaciones por categoría de forma automática sin necesidad de calcular las frecuencias por separado con `table()` y pasarlas después a un gráfico de barras manual.

El resultado muestra una distribución relativamente pareja entre los dos grupos, con un número algo mayor de URLs absolutas que relativas dentro de los 169 enlaces analizados. Ese equilibrio es consistente con lo observado en el Punto 5: buena parte de los enlaces relativos corresponden a navegación interna dentro del propio dominio de MediaWiki, mientras que los absolutos incluyen tanto enlaces internos escritos con el dominio completo como los enlaces externos hacia otros proyectos de Wikimedia.

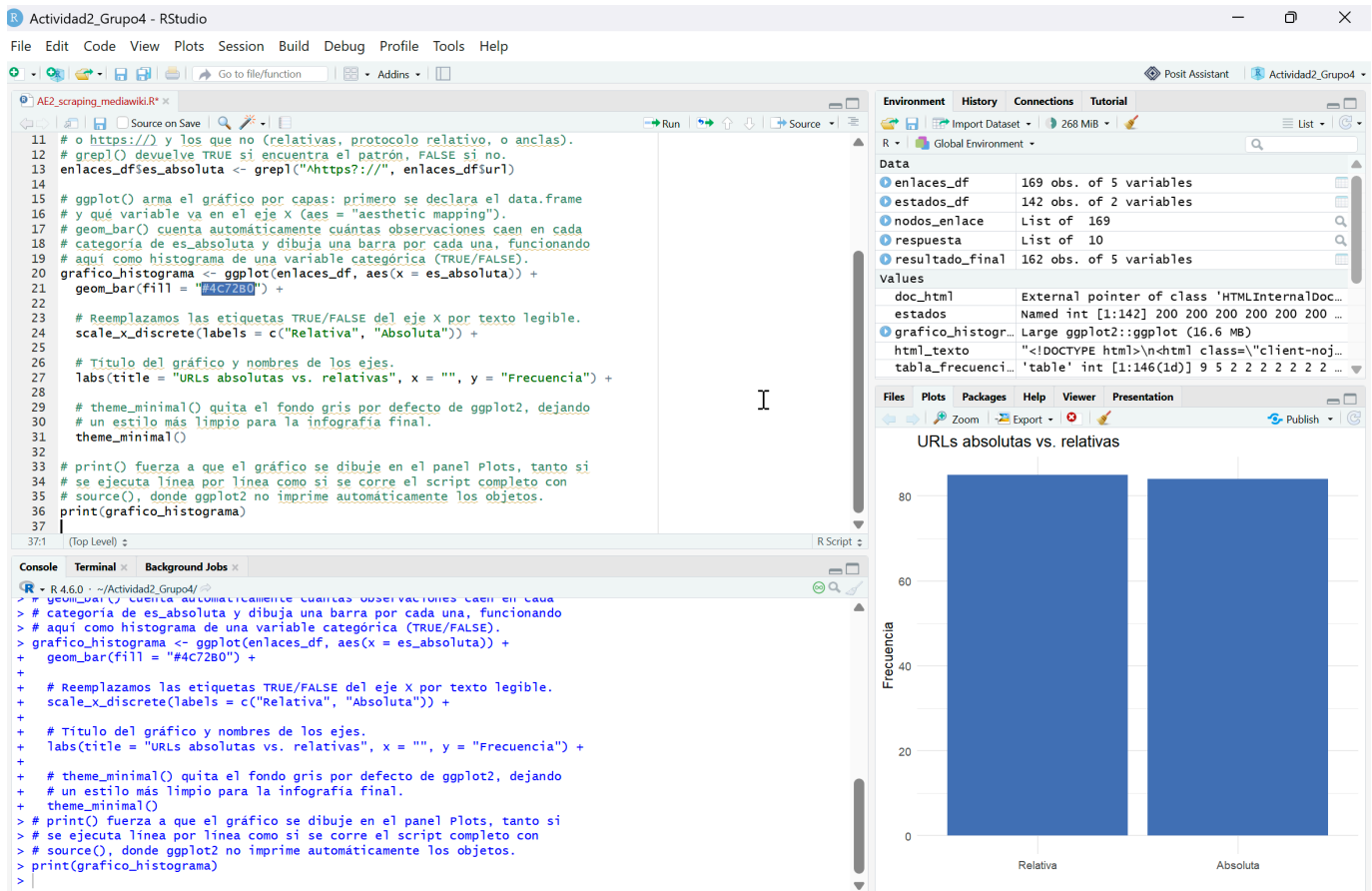


Imagen 8. Histograma: URLs absolutas vs. relativas

2. Gráfico de barras: enlaces internos vs. externos

Responsable: Maider Ibarra

Justificación del criterio de "enlace interno" (apunta a <https://www.mediawiki.org>) y del uso de `na.rm = TRUE`.

Código R:

```

# table() cuenta cuántas veces aparece cada valor de tipo_url (interno,
# externo, ancla), la misma columna que construimos en el Punto 5 al
# comparar cada URL normalizada contra el dominio base.
# useNA = "no" excluye del conteo cualquier valor NA que pudiera quedar
# en la columna (aquí no debería haber ninguno, pero se deja explícito
# por seguridad).

```

```

conteo_tipo <- table(enlaces_df$tipo_url, useNA = "no")

```

```

# as.data.frame() convierte la tabla de conteo en un data.frame de dos
# columnas (Var1 = categoría, Freq = frecuencia), formato que ggplot2
# necesita para poder mapear cada columna a un eje del gráfico.

```

```

grafico_barras <- ggplot(
  as.data.frame(conteo_tipo),
  aes(x = Var1, y = Freq, fill = Var1)
) +

```

```

# geom_col() dibuja una barra cuya altura es directamente el valor
# de la columna Freq (a diferencia de geom_bar(), que cuenta filas;
# aquí ya tenemos el conteo hecho por table()).
# na.rm = TRUE le indica a ggplot2 que, si existiera alguna fila con
# valor NA en x o y, la omite silenciosamente en vez de imprimir una
# advertencia por cada una.

```

```
geom_col(na.rm = TRUE) +  
  
labs(title = "Enlaces internos vs. externos", x = "", y = "Frecuencia") +  
theme_minimal() +  
  
# Quitamos la leyenda de colores: cada barra ya está etiquetada en  
# el eje X, así que la leyenda sería redundante.  
theme(legend.position = "none")  
  
# print() para que el gráfico se dibuje sin importar cómo se ejecute el script.  
print(grafico_barras)
```

Salida de consola:

```
> print(grafico_barras)  
(Se despliega en el panel Plots un gráfico de barras titulado  
"Enlaces internos vs. externos", con tres categorías en el eje X,  
ancla, externo e interno, y la frecuencia de cada una en el eje Y.  
La barra "interno" supera ampliamente a "externo", y "ancla" es  
la más baja de las tres)
```

Justificación:

El criterio para llamar interno a un enlace ya había quedado fijado en el Punto 5: una URL se considera interna cuando, tras normalizarla, empieza por el mismo dominio que se está analizando, <https://www.mediawiki.org>. Aquí ese trabajo no se repite, se reutiliza directamente la columna `tipo_url` calculada antes, y el gráfico se limita a mostrar cuántos enlaces caen en cada una de las tres categorías posibles, interno, externo y ancla.

`table(enlaces_df$tipo_url, useNA = "no")` hace el conteo, y `useNA = "no"` indica explícitamente que cualquier valor faltante en esa columna quede fuera del resultado. En la práctica no debería haber ningún NA en `tipo_url`, porque el Punto 5 asigna siempre una de las tres etiquetas a cada fila, pero dejarlo explícito documenta la intención y evita sorpresas si en algún momento cambia la lógica de clasificación.

`na.rm = TRUE` dentro de `geom_col()` cumple un papel distinto: no filtra los datos que entran al gráfico, sino que le indica a `ggplot2` que, si alguna barra terminara con un valor NA en la altura, la omita en silencio en vez de imprimir una advertencia por cada fila afectada. Es una precaución adicional sobre la misma idea, cubrir el caso de valores faltantes, aplicada en dos capas distintas: primero al construir la tabla de conteo y después al dibujarla.

El resultado confirma la proporción vista en el Punto 5: el grupo interno domina ampliamente sobre el externo y el de anclas, lo que muestra que la página analizada mantiene su navegación centrada en el propio dominio del wiki, con un número comparativamente pequeño de referencias a recursos externos.

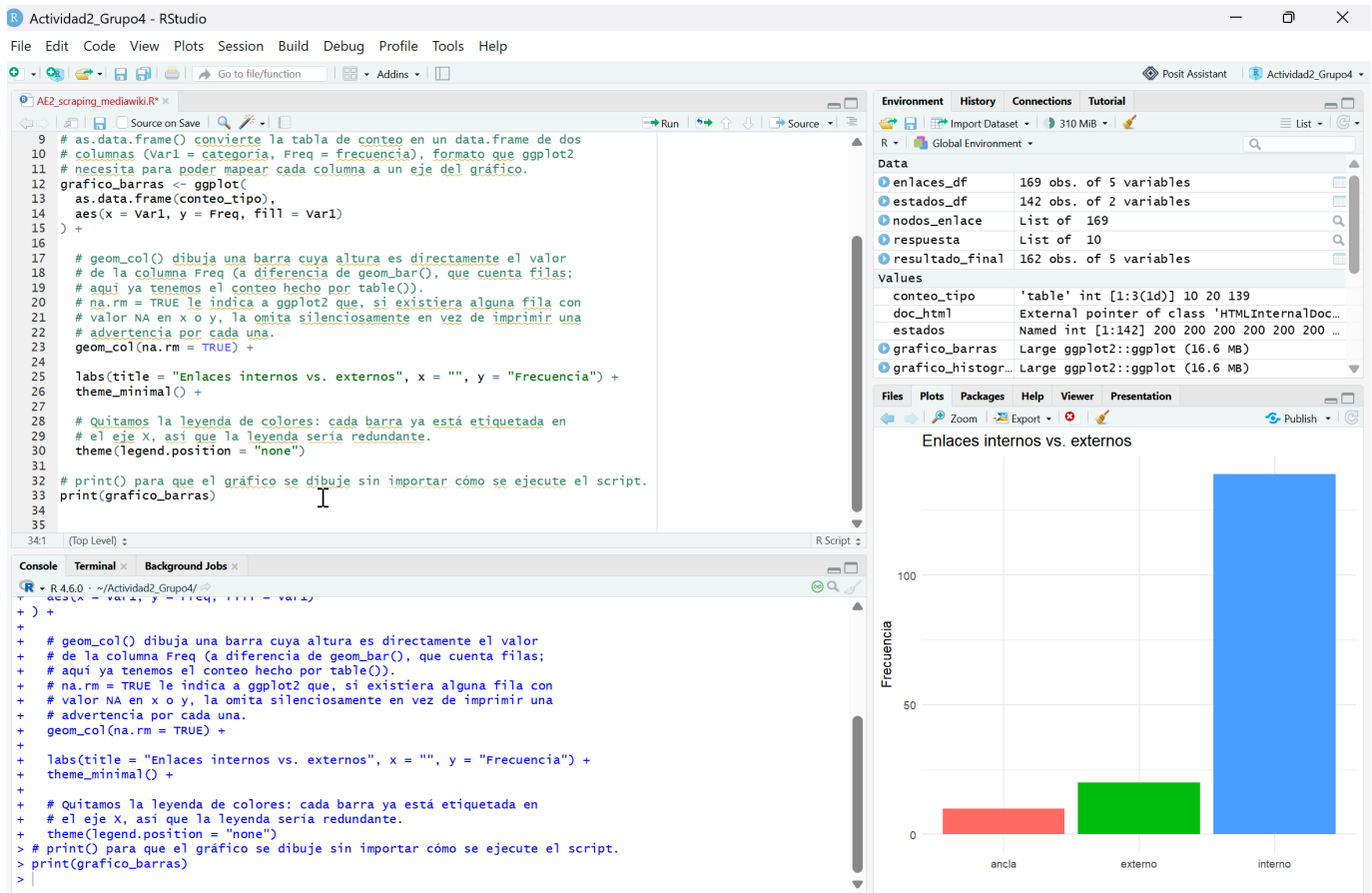


Imagen 9. Gráfico de barras: enlaces internos vs. externos

3. Gráfico de tarta: distribución de códigos de estado HTTP

Responsable: Carlos Noboa

Justificación del cálculo de porcentajes por status_code y su representación.

Código R:

```

# table() cuenta cuántas veces aparece cada código de estado HTTP.
# useNA = "ifany" incluye también los NA en el conteo (los enlaces
# que fallaron la consulta HEAD() por timeout u otro error), en vez
# de descartarlos silenciosamente como haría useNA = "no".
distribucion_estado <- as.data.frame(table(resultado_final$status_code, useNA = "ifany"))
names(distribucion_estado) <- c("status_code", "conteo")

# Reemplazamos la etiqueta NA (que ggplot2 mostraría en blanco o como
# <NA>) por un texto descriptivo, para que la leyenda del gráfico sea
# legible.
distribucion_estado$status_code <- as.character(distribucion_estado$status_code)
distribucion_estado$status_code[is.na(distribucion_estado$status_code)] <- "Error/Timeout"

# Calculamos el porcentaje que representa cada código de estado sobre
# el total de enlaces consultados, redondeado a un decimal.
distribucion_estado$porcentaje <- round(
  100 * distribucion_estado$conteo / sum(distribucion_estado$conteo), 1
)

print(distribucion_estado)

# Un gráfico de tarta en ggplot2 no existe como geometría propia:
# se construye "doblando" un gráfico de barras apiladas sobre un
# sistema de coordenadas polares.

```

```
grafico_tarta <- ggplot(distribucion_estado, aes(x = "", y = conteo, fill = status_code)) +

# geom_col(width = 1) crea una única barra apilada que ocupa todo
# el ancho disponible (width = 1), con un segmento por cada status_code.
geom_col(width = 1) +

# coord_polar("y") "envuelve" esa barra apilada alrededor de un círculo,
# usando el eje Y (los conteos) como ángulo de cada porción.
coord_polar("y") +

labs(title = "Distribucion de codigos de estado HTTP", fill = "Status") +

# theme_void() elimina ejes, cuadrícula y fondo, dejando solo el
# círculo y la leyenda, apropiado para un gráfico de tarta.
theme_void()

# print() para que el gráfico se dibuje sin importar cómo se ejecute el script.
print(grafico_tarta)
```

Salida de consola:

```
> print(distribucion_estado)
  status_code conteo porcentaje
1         200    157      96.9
2 Error/Timeout     5       3.1

> print(grafico_tarta)
(Se despliega en el panel Plots un gráfico circular con dos segmentos:
uno grande en rojo, "200" (96.9%), y uno pequeño en turquesa,
"Error/Timeout" (3.1%))
```

Justificación:

El cálculo de porcentajes se hizo dividiendo el conteo de cada código de estado entre la suma total de conteos, multiplicando por 100 y redondeando a un decimal con `round()`. Trabajar con porcentajes en vez de conteos absolutos facilita interpretar la proporción de cada categoría de un vistazo, algo que un gráfico de tarta comunica precisamente por el tamaño relativo de cada porción, no por el valor exacto de cada segmento.

La primera versión de este gráfico usaba `useNA = "no"` al construir la tabla, lo que excluía en silencio los enlaces cuya consulta `HEAD()` había fallado. Al revisar el resultado, con un solo color visible en el círculo, se detectó que había 5 enlaces con `status_code` en `NA` que no estaban entrando al conteo. Cambiar a `useNA = "ifany"` corrige eso, y renombrar ese valor `NA` como `Error/Timeout`, en lugar de dejarlo como una etiqueta vacía o `<NA>`, deja explícito en la leyenda qué representa ese segmento.

`ggplot2` no tiene una geometría dedicada para gráficos de tarta. La técnica habitual, y la que se usó aquí, consiste en dibujar primero una barra apilada de ancho fijo con `geom_col(width = 1)`, donde cada `status_code` ocupa un segmento proporcional a su conteo, y después envolver esa barra sobre coordenadas polares con `coord_polar("y")`, usando el eje Y como ángulo. `theme_void()` se añadió al final para quitar ejes y cuadrícula, dejando visible solo el círculo y la leyenda de colores.

El resultado muestra que 157 de los 162 enlaces consultados (96.9%) respondieron con código 200, y solo 5 (3.1%) fallaron la consulta por timeout u otro error de red. Esa proporción indica que, en el momento del análisis, la gran mayoría de los recursos enlazados desde la página de MediaWiki estaban disponibles y accesibles.

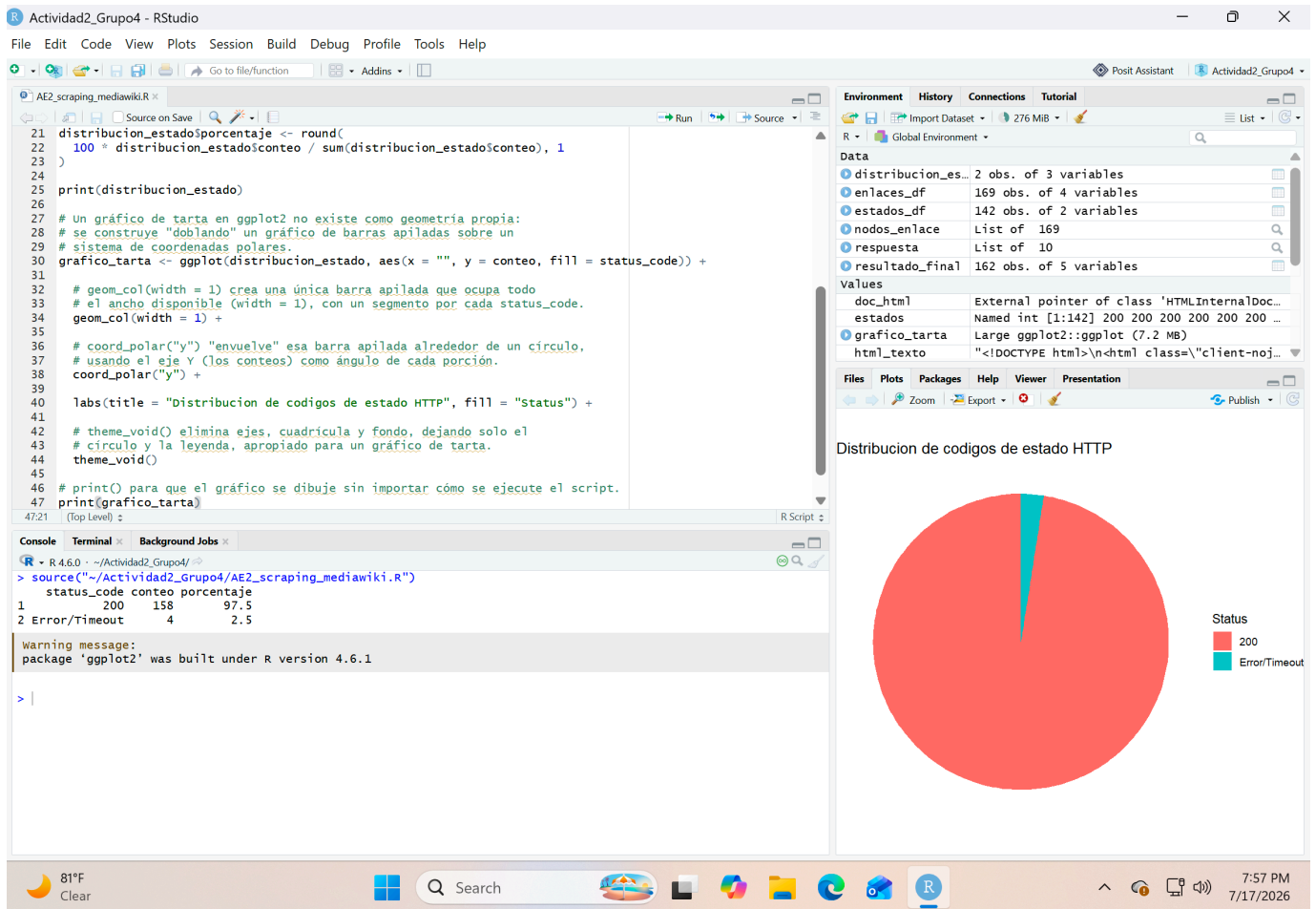


Imagen 10. Gráfico de tarta: distribución de códigos de estado HTTP

Composición final de la infografía

Responsable: Camilo Arias

Justificación del método usado para unir los tres gráficos en una sola figura (par) o librería de composición como gridExtra/patchwork).

Código R:

```

# Carga gridExtra, la librería de composición que combina varios objetos
# ggplot2 en una sola figura (ya se instaló junto con ggplot2 al inicio
# de la Pregunta 2).
library(gridExtra)

# grid.arrange() organiza varios gráficos ggplot2 en una cuadrícula.
# ncol = 3 los coloca uno al lado del otro, en el mismo orden en que
# se pasan como argumentos: histograma, barras, tarta.
infografia_final <- grid.arrange(
  grafico_histograma, grafico_barras, grafico_tarta,
  ncol = 3
)

# ggsave() guarda la última figura dibujada en un archivo de imagen.
# width y height (en pulgadas) se ajustan para que los tres gráficos
# se vean bien proporcionados uno junto al otro en formato horizontal.
ggsave("infografia_mediawiki.png", plot = infografia_final, width = 12, height = 4)

```

Salida de consola:

```
> library(gridExtra)
> infografia_final <- grid.arrange(
+   grafico_histograma, grafico_barras, grafico_tarta,
+   ncol = 3
+ )
(Se despliega en el panel Plots una única figura con los tres gráficos  
lado a lado: histograma de URLs absolutas/relativas, barras de enlaces  
internos/externos, y tarta de códigos de estado HTTP)

> ggsave("infografia_mediawiki.png", plot = infografia_final, width = 12, height = 4)
```

Justificación:

Para unir los tres gráficos se recurrió a gridExtra en lugar de la función par(mfrow = ...) de gráficos base de R. par() organiza en una cuadrícula los gráficos que se van dibujando sucesivamente con funciones como plot() o hist(), pero no está pensada para combinar objetos ggplot2, que se construyen y se imprimen de otra manera. gridExtra sí sabe tratar un objeto ggplot como una unidad completa y colocarlo dentro de una cuadrícula junto a otros del mismo tipo.

grid.arrange() recibe los tres gráficos ya contruidos, grafico_histograma, grafico_barras y grafico_tarta, y el argumento ncol = 3 fija que se dispongan en una sola fila de tres columnas. El orden en que se pasan como argumentos determina el orden de izquierda a derecha en la figura final, por eso se mantuvo la misma secuencia en que se fueron desarrollando en el informe.

A diferencia de un objeto ggplot individual, grid.arrange() devuelve un gtable ya renderizado, no una receta pendiente de dibujar. Por eso, cuando se corre esta línea, la figura combinada aparece de inmediato en el panel Plots sin necesidad de envolverla en print(), algo que sí hacía falta en los tres gráficos individuales anteriores.

El último paso, ggsave(), guarda esa misma figura en un archivo PNG con dimensiones de 12 por 4 pulgadas, un formato apaisado que da espacio suficiente para que los tres gráficos se lean con claridad uno junto al otro sin quedar demasiado angostos. Ese archivo, infografia_mediawiki.png, es el que se entrega junto con el script como evidencia visual independiente del análisis completo.

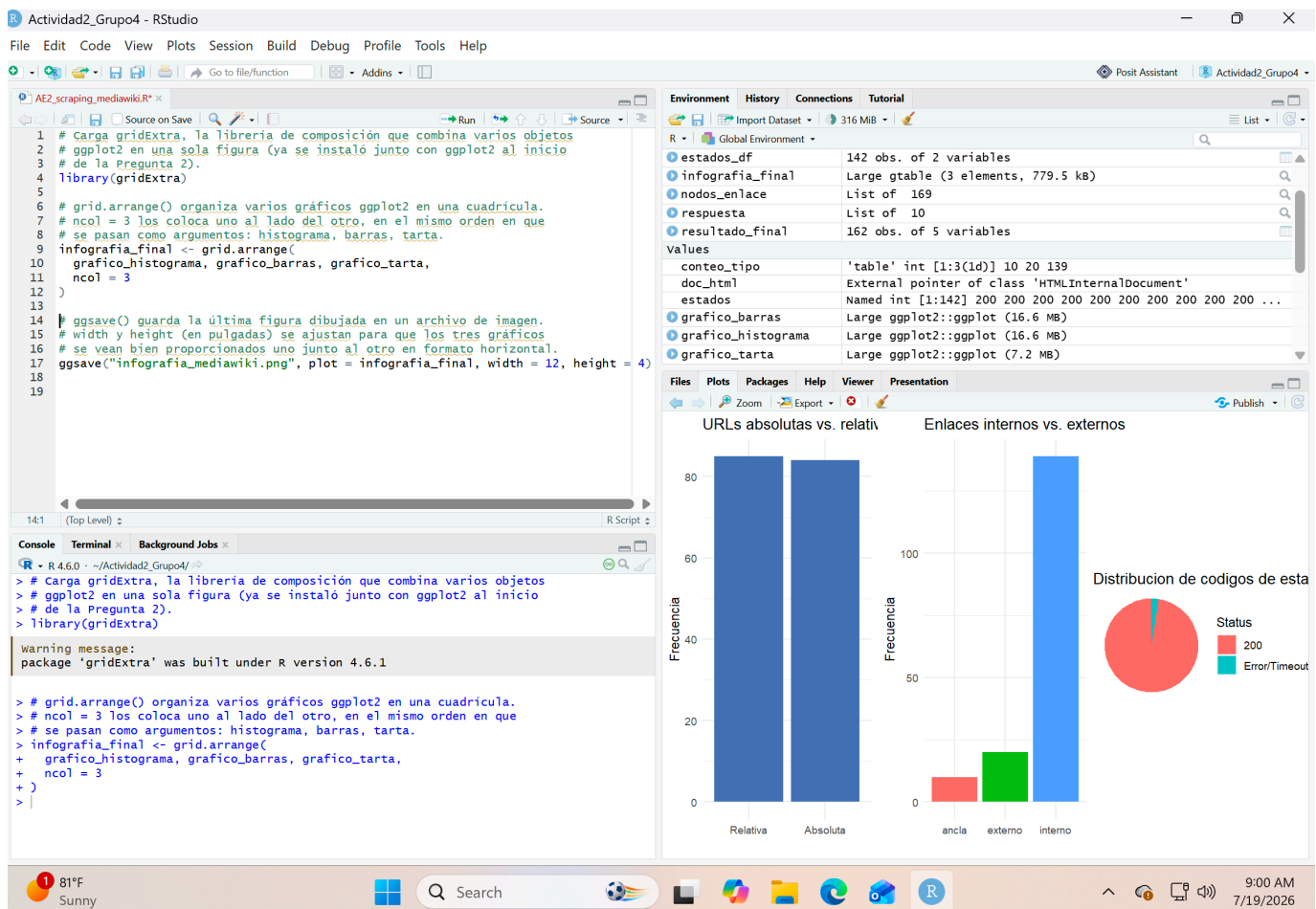


Imagen 11. Composición final de la infografía

2. Conclusiones

Resumen breve de las dificultades encontradas, decisiones de diseño relevantes y aprendizajes del grupo durante el desarrollo de la actividad.

El desarrollo de la actividad reveló varias diferencias entre correr el código línea por línea durante las pruebas y ejecutarlo de un tirón con `source()`, como exige la entrega final. Los tres gráficos de ggplot2 se construyeron sin problema en ambos casos, pero solo se dibujaban en pantalla al escribirlos directamente en la consola; dentro de un script fuente, un objeto ggplot no se imprime automáticamente. La solución fue envolver cada gráfico en `print()`, un ajuste sencillo una vez identificado, aunque su ausencia inicial generó confusión porque el objeto se creaba correctamente en el Environment sin que apareciera ningún error.

Algo parecido ocurrió al verificar que el script corriera en un terminal limpio. Reiniciar la sesión de R con `Session > Restart R` no bastó por sí solo, porque RStudio recupera automáticamente los objetos guardados en un `.RData` previo si esa opción sigue activa. Confirmar una prueba realmente representativa exigió además vaciar el Environment a mano antes de correr `source()`, un paso que no era obvio al principio pero que resultó necesario para sostener la afirmación del enunciado sobre ausencia de dependencias previas.

Un tercer punto de fricción tuvo que ver con los datos, no con el código en sí. El gráfico de tarta de códigos de estado HTTP mostraba inicialmente un solo color, y la primera hipótesis fue que se trataba de un error de programación. Revisando la tabla de frecuencias con `useNA = "ifany"` en vez de `"no"`, se confirmó que 5 de los 162 enlaces habían fallado la consulta `HEAD()` por timeout, y que la opción original simplemente los estaba excluyendo del conteo sin avisar. El ajuste no cambió la lógica del script, corrigió cómo se contaban los valores faltantes antes de graficarlos.

Entre las decisiones de diseño que se sostuvieron durante todo el desarrollo está la normalización previa de URLs en categorías (interno, externo, ancla) antes de intentar consultar su estado HTTP, lo que evitó desperdiciar peticiones de red sobre anclas internas que nunca iban a resolver a un recurso distinto. El uso de HEAD() en lugar de GET() para la verificación de estado, junto con Sys.sleep(0.5) entre cada consulta, mantuvo el scraping dentro de un margen razonable de tiempo sin sobrecargar el servidor de MediaWiki.

Como aprendizaje general, el ejercicio dejó claro que verificar la reproducibilidad de un script exige algo más que revisarlo visualmente: hace falta correrlo de verdad en las condiciones que va a enfrentar quien lo evalúe, con el entorno vacío y sin asumir que los paquetes ya están cargados de una sesión anterior. También quedó patente el valor de mostrar explícitamente los valores faltantes en vez de descartarlos en silencio, una decisión que en este caso cambió por completo la lectura visual de uno de los tres gráficos de la infografía.

3. Bibliografía

R Core Team. (2026). R: A language and environment for statistical computing. R Foundation for Statistical Computing. <https://www.R-project.org/>

RStudio Team. (2024). RStudio: Integrated development environment for R. Posit Software, PBC. <https://posit.co/>

Wickham, H., Hester, J., & Bryan, J. (2023). httr: Tools for working with URLs and HTTP (R package). <https://httr.r-lib.org/>

Lang, D. T., & CRAN Team. (2023). XML: Tools for parsing and generating XML within R and S-Plus (R package). <https://CRAN.R-project.org/package=XML>

Wickham, H. (2016). ggplot2: Elegant graphics for data analysis. Springer-Verlag New York. <https://ggplot2.tidyverse.org/>

Auguie, B. (2017). gridExtra: Miscellaneous functions for "Grid" graphics (R package). <https://CRAN.R-project.org/package=gridExtra>

Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and content (RFC 7231). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7231>

Wikimedia Foundation. (s.f.). MediaWiki. <https://www.mediawiki.org/wiki/MediaWiki>